

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Facultad de Ciencias Puras · Carrera de Ingeniería Informática
Potosí — Bolivia

INF 220 — PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

GUÍA DE LABORATORIO N.º 3

Constructores y encapsulamiento

Unidad	Unidad 1 — Programación Orientada a Objetos
Tema	Encapsulamiento · modificadores de acceso · getters y setters · constructores y su sobrecarga
Requisito	Guía de Laboratorio N.º 2 (clases y objetos)
Stack	JDK Temurin 25 (LTS) · Visual Studio Code · Extension Pack for Java
Duración	1 sesión de laboratorio (90 min)
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

1 Objetivos

► Objetivo general

Aplicar el encapsulamiento para proteger el estado de los objetos y emplear constructores para inicializarlos de forma correcta y controlada.

► Objetivos específicos

- Declarar atributos como `private` y exponerlos mediante métodos `getter` y `setter`.
- Reconocer y aplicar los modificadores de acceso de Java.
- Incluir validaciones en los `setters` para mantener el estado del objeto consistente.
- Implementar constructores para inicializar los objetos al crearlos.
- Sobrecargar constructores para ofrecer distintas formas de crear un objeto.

2 Requisitos previos

Software	JDK Temurin 25 + Visual Studio Code con Extension Pack for Java.
Conocimientos	Definición de clases con atributos y métodos, instanciación con <code>new</code> y uso de <code>this</code> (Guía 2).

3 Marco teórico

► Encapsulamiento: proteger el estado

En la Guía 2 los atributos eran accesibles desde cualquier lugar con `e1.edad = -5;`. Nada impedía asignar valores inválidos. El **encapsulamiento** consiste en ocultar los atributos y permitir el acceso a ellos solo a través de métodos controlados, de modo que el objeto proteja su propio estado.

DEFINICIÓN • Encapsulamiento

Se declaran los atributos como **private** (nadie fuera de la clase los toca directamente) y se ofrecen métodos públicos para leerlos y modificarlos de forma segura.

► Modificadores de acceso

private	Solo accesible dentro de la propia clase. Es lo habitual para los atributos.
public	Accesible desde cualquier clase. Se usa para los métodos que forman la interfaz del objeto.
protected	Accesible dentro del paquete y por las clases hijas (se verá con la herencia).
(default)	Sin modificador: accesible solo dentro del mismo paquete.

► Getters y setters

Son los métodos que dan acceso controlado a un atributo privado. Por convención, `getX()` devuelve el valor y `setX(valor)` lo modifica. El **setter** es el lugar ideal para validar antes de asignar.

```
private double promedio;

public double getPromedio() {
    return promedio;
}
public void setPromedio(double promedio) {
    this.promedio = promedio; // this distingue el atributo del parámetro
}
```

► Constructores

Un **constructor** es un método especial que se ejecuta automáticamente al crear el objeto con `new`. Tiene el mismo nombre que la clase y no declara tipo de retorno. Sirve para dejar el objeto listo y con valores iniciales válidos.

```
public Estudiante(String nombre, int edad, double promedio) {
    this.nombre = nombre;
    this.edad = edad;
    this.promedio = promedio;
}
```

NOTA • El constructor por defecto

Si no escribís ningún constructor, Java te da uno vacío automáticamente. Pero en cuanto definís uno con parámetros, ese constructor por defecto **desaparece**: si todavía necesitás crear objetos sin argumentos, deberás declararlo vos mismo.

► Sobrecarga de constructores

Una clase puede tener **varios** constructores, siempre que se diferencien en el número o el tipo de sus parámetros. Así se ofrecen distintas maneras de crear el objeto según los datos disponibles.

4 Desarrollo de la práctica

Trabaja en una carpeta Lab03. Partimos de la clase Estudiante de la Guía 2 y la vamos transformando.

► Actividad 1 — Encapsular la clase Estudiante

Declara los atributos como `private` y agrega los getters y setters correspondientes:

```
public class Estudiante {
    private String nombre;
    private int edad;
    private double promedio;

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }

    public double getPromedio() { return promedio; }
    public void setPromedio(double promedio) { this.promedio = promedio; }
}
```

Desde `Principal.java`, ahora el acceso es a través de los métodos:

```
Estudiante e1 = new Estudiante();
e1.setNombre("Ana");
e1.setEdad(20);
e1.setPromedio(78.5);
System.out.println(e1.getNombre() + " - " + e1.getPromedio());
```

EJEMPLO · Comprueba la protección

Intenta escribir `e1.edad = 20;` directamente: el compilador ahora lo marca como error, porque `edad` es `private`. Ese es el encapsulamiento funcionando.

► Actividad 2 — Validar en los setters

Modifica los setters para que rechacen valores fuera de rango, protegiendo el estado del objeto:

```
public void setEdad(int edad) {
    if (edad >= 0 && edad <= 120) {
        this.edad = edad;
    } else {
        System.out.println("Edad inválida, se ignora el valor.");
    }
}

public void setPromedio(double promedio) {
    if (promedio >= 0 && promedio <= 100) {
        this.promedio = promedio;
    } else {
        System.out.println("Promedio fuera de rango (0-100).");
    }
}
```

```
}
```

Prueba asignar un valor inválido como `e1.setEdad(-5)`; y observa que el objeto no cambia su estado.

► Actividad 3 — Constructor con parámetros

Agrega un constructor que reciba los tres datos e inicialice el objeto al crearlo:

```
public Estudiante(String nombre, int edad, double promedio) {
    this.nombre = nombre;
    this.edad = edad;
    this.promedio = promedio;
}
```

Ahora el objeto nace ya inicializado en una sola línea:

```
Estudiante e1 = new Estudiante("Ana", 20, 78.5);
e1.mostrarDatos();
```

► Actividad 4 — Sobrecargar constructores

Ofrece tres formas de crear un estudiante añadiendo dos constructores más:

```
public Estudiante() {
    // objeto vacío, se completa luego con los setters
}

public Estudiante(String nombre) {
    this.nombre = nombre;
}

public Estudiante(String nombre, int edad, double promedio) {
    this.nombre = nombre;
    this.edad = edad;
    this.promedio = promedio;
}
```

NOTA · Reutilizar constructores con `this(...)`

Un constructor puede llamar a otro de la misma clase con `this(...)` en su primera línea, para no repetir código. Por ejemplo, el constructor de un solo dato podría delegar en el completo.

► Actividad 5 — Aplicar todo: CuentaBancaria

Crea una clase encapsulada con constructor y métodos que protejan las operaciones:

```
public class CuentaBancaria {
    private String titular;
    private double saldo;

    public CuentaBancaria(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public void depositar(double monto) {
        if (monto > 0) { saldo += monto; }
    }
}
```

```
    }

    public void retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
        } else {
            System.out.println("Operación inválida.");
        }
    }

    public double getSaldo() { return saldo; }
}
```

Pruébala creando una cuenta, depositando y retirando, y verificando el saldo tras cada operación.

5 Bibliografía y recursos

- Deitel, P. & Deitel, H. — Cómo programar en Java. Pearson.
- Sierra, K. & Bates, B. — Head First Java. O'Reilly.
- Encapsulamiento y constructores (tutorial oficial):
<https://docs.oracle.com/javase/tutorial/java/javaOO/>